

Compiler Construction

⌞ Type Systems ⌞

Why we don't need types

Consider the MIPS ASM fragment:

```
add $1, $2, $3
```

Types are not necessary! Assembly language is untyped

- There is none at machine/assembly level. Operators are “typed” though
- There are type-less languages e.g., in Tcl or M4 everything is a string

Why we need types

- It makes sense to add two integers
- It does not make sense to add an integer to a function pointer in C
- But both have the same assembly language implementation!

Type-checking

detect and reject meaningless operations instead of applying them silently

- compile-time detection → statically typed language
- runtime detection → dynamically typed language

Goal of a Type System

Reject impossible/meaningless values

Static Typing vs. Dynamic Typing

- **Statically typed languages:** all or almost all type checking occurs at compilation time.
 - ▶ OCaml, C, Java, etc.
- **Dynamically typed languages:** almost all checking of types is done as part of program execution.
 - ▶ Scheme, Python, etc.
- **Untyped languages:** no type checking
 - ▶ Assembly, Machine code

Strong vs Weak Typing - 1

Strong $\neq$ Static

Strong

If a program is accepted by the type system, it won't fail due to a type error.

All type errors are detected by the type system.

But sometimes correct programs are rejected too

```
(* ocaml *)  
if true then 42 else "i'm not an int :("  
(* Error: This expression has type string but an expression was  
   expected of type int *)
```

Most of the time, strong $\Rightarrow$ static

Strong vs Weak Typing - 2

Weak $\neq$ Dynamic

Weak

If a program is rejected by the type system, its execution (very likely) causes a type error.
Only type errors are detected by the type system

But sometimes erroneous programs are accepted !

```
// java
Object i = Integer.valueOf(42);
String s = (String)i;           // ClassCastException thrown here.
```

Most of the time, dynamic $\Rightarrow$ weak

Explicit Vs Implicit

When we specify the type of an entity by explicitly stating it in the source code, we are doing explicit typing. Variables and functions in C, and Java is mostly explicit.

```
// java
public int f (int x) {
    return x + 42;
}
```

When types are not specified explicitly (e.g., OCaml, Python, JavaScript), we have implicit typing.

```
(* ocaml *)
let f x = x + 42
```


ILP's type system

Is ILP's type system:

- Static or Dynamic?
- Explicit or Implicit?
- Strong or Weak?

ILP's type system

Is ILP's type system:

- Static or Dynamic?
- Explicit or Implicit?
- Strong or Weak?

ILP is a dynamically, implicitly, strongly typed language.

Types in a Compiler?

Dynamic typing (runtime overhead):

```
if (ILP_isInteger(o1) && ILP_isInteger(o2)) {  
    ILP_Object result = ILP_AllocateInteger();  
    result->_content.asInteger =  
        o1->_content.asInteger + o2->_content.asInteger;  
    return result;  
}
```

Static typing (compile-time certainty):

`o1 + o2`

Types known at compile time eliminate checks, boxing, and indirections.

A Static Type System for ILP

Exploiting type information at compile-time make it possible to generate :

- faster code
- simpler code
- safer code

We are going to develop a static type system for ILP

But ILP has no explicit type!

No problem, we can infer them (in most of the time)

Types

The data types of a language are a large part of what determines that language's style and usefulness (along with control structures).

- **Primitive (built-in) types:** data type provided by a programming language.
ILP has bool, int, float, string, functions
- **Composite types:** recursively constructed starting from primitive types
ILP has functions and objects

Type Checking

Type Checking is the activity of ensuring that the operands of an **operator** are of compatible types

A **compatible type** is one that is

- either legal for the operator
- **or** allowed under language rules to be implicitly converted by compiler-generated code (or the interpreter) to a legal type

Type Checking and Type Inference

- Type Checking is the process of verifying fully typed programs
- Type Inference is the process of filling in missing type information

Remark

- Type Checking generally some degree of inference
- The two are different, but are often confused!

Type Checking in Practice

Type checking is done compositionally:

- 1 break down expressions into their subexpressions
- 2 type-check each subexpression
- 3 ensure that the top-level compound expression can then be given a type itself

Throughout the process, a type environment is maintained which records the types of all variables in the expression.

Inference Rule

- **Types do not need to be explicit to have static typing.**
With inference rules, we can infer types!
- We will use an appropriate formalism to express inference rules!
 - ▶ With a good notation, we can check the accuracy of the rules;
 - ▶ With a good notation, we can easily translate it into programs.

From English to an Inference Rule

If e_1 has type Int and e_2 has type Int
then $e_1 + e_2$ has type Int .

From English to an Inference Rule

If e_1 has type Int and e_2 has type Int
then $e_1 + e_2$ has type Int .

$(e_1 \text{ has type } Int \wedge e_2 \text{ has type } Int)$
 $\Rightarrow e_1 + e_2 \text{ has type } Int.$

From English to an Inference Rule

If e_1 has type Int and e_2 has type Int
then $e_1 + e_2$ has type Int .

$(e_1 \text{ has type } Int \wedge e_2 \text{ has type } Int)$
 $\Rightarrow e_1 + e_2 \text{ has type } Int.$

$(e_1: Int \wedge e_2: Int)$
 $\Rightarrow e_1 + e_2: Int.$

Generalization

The statement:

- $(e_1: \text{Int} \wedge e_2: \text{Int}) \implies e_1 + e_2: \text{Int}.$

...is a special case of:

- $(\text{Hypothesis}_1: \text{Int} \wedge \dots \wedge \text{Hypothesis}_n: \text{Int}) \implies \text{Conclusion}$

This is an inference rule!

Notation

By tradition inferences rules are written:

$$\frac{\vdash \text{Hyp}_1 \quad \dots \quad \vdash \text{Hyp}_n}{\text{Conclusion}}$$

$\vdash$ means "*it is provable that...*"

Formalization of ILP1's type system.

Typing Rules

Detect the type of a literal:

$$\frac{\vdash i \text{ is an integer literal}}{\vdash i: \text{int}}$$

Typing Rules

Detect the type of a literal:

$$\frac{\vdash i \text{ is an integer literal}}{\vdash i: \text{int}}$$

Determining the type of a binary operation, depends on the type of the operands ...

$$\frac{\vdash e_1 : \text{int} \quad \vdash e_2 : \text{int}}{\vdash e_1 + e_2 : \text{int}}$$

Typing Rules

Detect the type of a literal:

$$\frac{\vdash i \text{ is an integer literal}}{\vdash i: \text{int}}$$

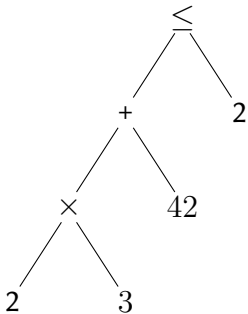
Determining the type of a binary operation, depends on the type of the operands ...

$$\frac{\vdash e_1: \text{int} \quad \vdash e_2: \text{int}}{\vdash e_1 + e_2: \text{int}}$$

...and the operator!

$$\frac{\vdash e_1: \text{int} \quad \vdash e_2: \text{int}}{\vdash e_1 < e_2: \text{bool}}$$

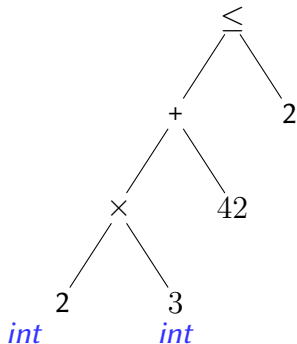
Type Inference Example - 1



Type inference is a bottom-up traversal of the AST:

- The type of each node depends of the type of its children
- We must know the type of leafs

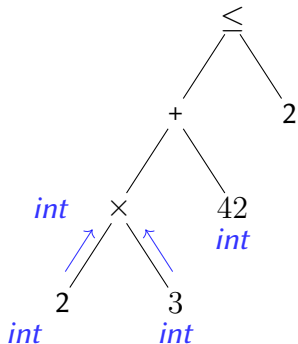
Type Inference Example - 1



Type inference is a bottom-up traversal of the AST:

- The type of each node depends of the type of its children
- We must know the type of leafs

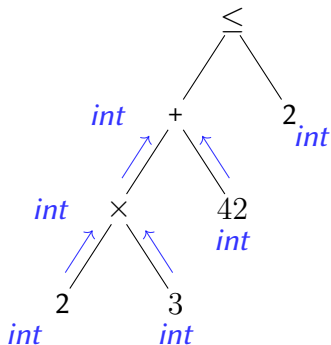
Type Inference Example - 1



Type inference is a bottom-up traversal of the AST:

- The type of each node depends of the type of its children
- We must know the type of leafs

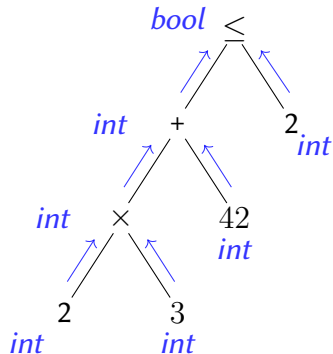
Type Inference Example - 1



Type inference is a bottom-up traversal of the AST:

- The type of each node depends of the type of its children
- We must know the type of leafs

Type Inference Example - 1



Type inference is a bottom-up traversal of the AST:

- The type of each node depends of the type of its children
- We must know the type of leafs

Type of Variables

Variables do not have intrinsic types. Their type is determined by their *most recent binding* $\Rightarrow$ **we need a typing environment Γ mapping variables to types.**

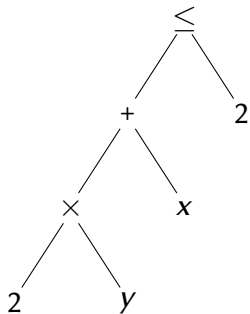
$$\frac{x : \tau \in \Gamma}{\Gamma \vdash x : \tau}$$

Where types come from

- Block bindings: typed from their initialisation
- Assignments give the left value the type of the right value (ILP2)
- Function parameters (ILP2)

Remarks: Variables may change type over time

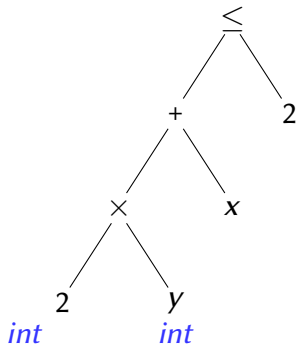
Type Inference Example - 1



The environment Γ :

- ...
- $x \rightarrow \text{int}$
- $y \rightarrow \text{int}$
- ...

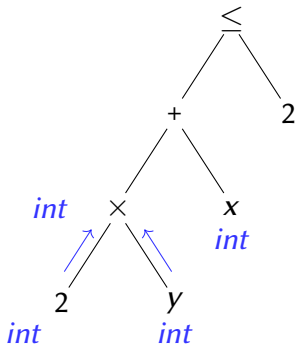
Type Inference Example - 1



The environment Γ :

- ...
- $x \rightarrow \text{int}$
- $y \rightarrow \text{int}$
- ...

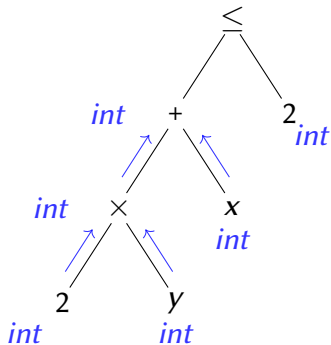
Type Inference Example - 1



The environment Γ :

- ...
- $x \rightarrow \text{int}$
- $y \rightarrow \text{int}$
- ...

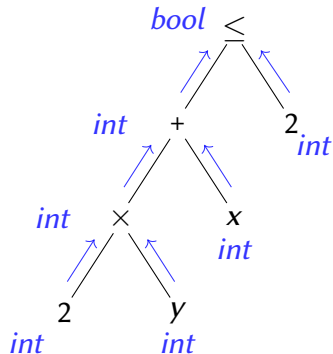
Type Inference Example - 1



The environment Γ :

- ...
- $x \rightarrow \text{int}$
- $y \rightarrow \text{int}$
- ...

Type Inference Example - 1



The environment Γ :

- ...
- $x \rightarrow int$
- $y \rightarrow int$
- ...

Coercion

Coercion is the automatic (implicit) conversion from a type to another.

There are 2 kind of coercions:

- **widening** conversions: from a "smaller" type to a "larger one"

```
int i = 42;  
float f = i;
```

- **narrowing** conversions: from a "larger" type to a "smaller one"

```
float f = 42.0;  
int i = f;
```

In ILP, each operator has its own coercion rules!

Binary Arithmetic Operations

When $\diamond \in \{+, -, /, *\}$

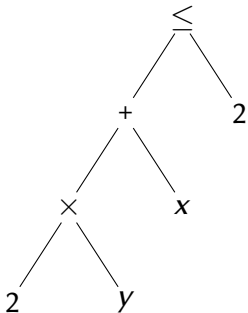
$$\frac{\vdash e_1 : \text{int} \quad \vdash e_2 : \text{int}}{\vdash e_1 \diamond e_2 : \text{int}}$$

$$\frac{\vdash e_1 : \text{float} \quad \vdash e_2 : \text{int}}{\vdash e_1 \diamond e_2 : \text{float}}$$

$$\frac{\vdash e_1 : \text{int} \quad \vdash e_2 : \text{float}}{\vdash e_1 \diamond e_2 : \text{float}}$$

$$\frac{\vdash e_1 : \text{float} \quad \vdash e_2 : \text{float}}{\vdash e_1 \diamond e_2 : \text{float}}$$

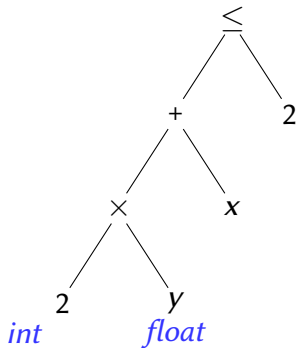
Type Inference Example - 2



The environment Γ :

- ...
- $x \rightarrow \text{int}$
- $y \rightarrow \mathbf{float}$
- ...

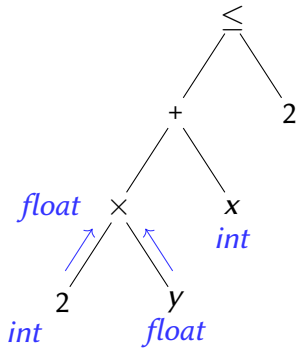
Type Inference Example - 2



The environment Γ :

- ...
- $x \rightarrow \text{int}$
- $y \rightarrow \mathbf{float}$
- ...

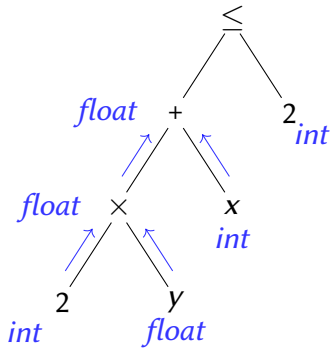
Type Inference Example - 2



The environment Γ :

- ...
- $x \rightarrow \text{int}$
- $y \rightarrow \mathbf{float}$
- ...

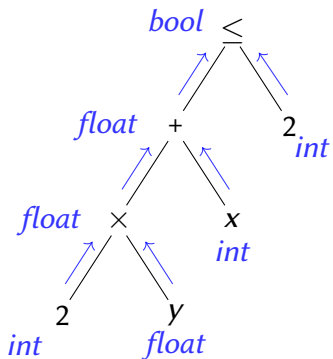
Type Inference Example - 2



The environment Γ :

- ...
- $x \rightarrow \text{int}$
- $y \rightarrow \mathbf{float}$
- ...

Type Inference Example - 2



The environment Γ :

- ...
- $x \rightarrow \text{int}$
- $y \rightarrow \mathbf{float}$
- ...

Special Case

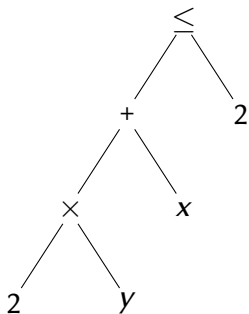
String Concatenation using +

$$\frac{\vdash e_1 : \text{string} \quad \vdash e_2 : ?}{\vdash e_1 + e_2 : \text{string}}$$

$$\frac{\vdash e_1 : ? \quad \vdash e_2 : \text{string}}{\vdash e_1 + e_2 : \text{string}}$$

Implicit conversion of operand to string

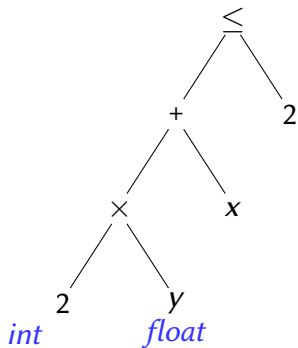
Type Inference Example - 3



The environment Γ :

- ...
- $x \rightarrow \mathbf{string}$
- $y \rightarrow \mathbf{float}$
- ...

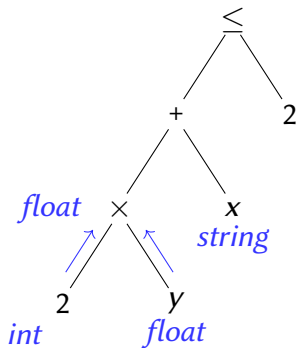
Type Inference Example - 3



The environment Γ :

- ...
- $x \rightarrow \mathbf{string}$
- $y \rightarrow \mathbf{float}$
- ...

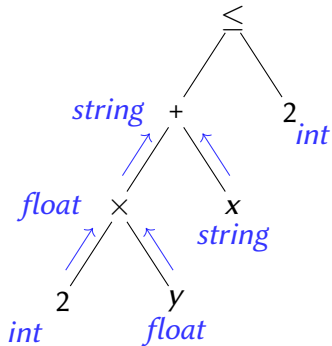
Type Inference Example - 3



The environment Γ :

- ...
- $x \rightarrow \mathbf{string}$
- $y \rightarrow \mathbf{float}$
- ...

Type Inference Example - 3

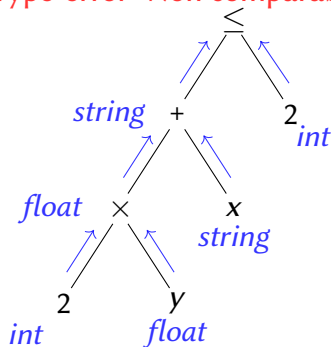


The environment Γ :

- ...
- $x \rightarrow \mathbf{string}$
- $y \rightarrow \mathbf{float}$
- ...

Type Inference Example - 3

Type-error: Non comparable



The environment Γ :

- ...
- $x \rightarrow \mathbf{string}$
- $y \rightarrow \mathbf{float}$
- ...

Inequalities

When $\diamond \in \{<, \leq, >, \geq\}$

$$\frac{\vdash e_1 : \text{int} \quad \vdash e_2 : \text{int}}{\vdash e_1 \diamond e_2 : \text{bool}}$$

$$\frac{\vdash e_1 : \text{float} \quad \vdash e_2 : \text{int}}{\vdash e_1 \diamond e_2 : \text{bool}}$$

$$\frac{\vdash e_1 : \text{int} \quad \vdash e_2 : \text{float}}{\vdash e_1 \diamond e_2 : \text{bool}}$$

$$\frac{\vdash e_1 : \text{float} \quad \vdash e_2 : \text{float}}{\vdash e_1 \diamond e_2 : \text{bool}}$$

$$\frac{\vdash e_1 : \text{string} \quad \vdash e_2 : \text{string}}{\vdash e_1 \diamond e_2 : \text{bool}}$$

Equalities

When $\diamond \in \{=, \neq\}$

$$\frac{\vdash e_1 : ? \quad \vdash e_2 : ?}{\vdash e_1 \diamond e_2 : \text{bool}}$$

$$\frac{\vdash e : ?}{\vdash \text{not } e : \text{bool}}$$

There are other operators (and, or ...), feel free to look at
ilp1/interpreter/operator/ to guess their typing rules

Sequence

- Follows the interpreter: return type is the type of the returned value.
- Return value is the value of the last expression

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \dots \quad \Gamma \vdash e_n : \tau_n}{\Gamma \vdash (e_1; \dots; e_n) : \tau_n}$$

Warning

Eventhough we dont care about $\tau_1, \dots, \tau_{n-1}$, the typing algorithm must visit them to give them type and detect errors!

Block with Simultaneous Bindings

Bindings are **simultaneous** (no binding sees another).

Typing rule

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \dots \quad \Gamma \vdash e_n : \tau_n \quad \Gamma, x_1 : \tau_1, \dots, x_n : \tau_n \vdash e : \tau}{\Gamma \vdash \{ x_1 = e_1; \dots; x_n = e_n; e \} : \tau}$$

Remarks

- All initialisations are type-checked in the **old environment** Γ
- The body is type-checked in the **extended environment** $\Gamma, x_1 : \tau_1, \dots, x_n : \tau_n$.

If-then-else (with unification)

$$\frac{\Gamma \vdash c : ? \quad \Gamma \vdash e_1 : \tau_1 \quad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash \text{if } c \text{ then } e_1 \text{ else } e_2 : \text{unify}(\tau_1, \tau_2)}$$

Remarks:

- Condition is not required to be `bool` here
- every ILP value **implicitly** has a truth value.
- We'll make this conversion explicit in a later stage

Type Unification - 1

Unification computes a single type (hopefully the smallest) that is compatible with its operand

What is the type of:

`if cond then 2 else 2.5 ?`

Type Unification - 1

Unification computes a single type (hopefully the smallest) that is compatible with its operand

What is the type of:

`if cond then 2 else 2.5 ?`

$\text{unify}(\tau_1, \tau_2)$

- τ_1 when $\tau_1 = \tau_2$
- FLOAT when $\tau_1 = \text{FLOAT} \ \& \ \tau_2 = \text{INT}$ (symmetric)

Remark: This may deviate slightly from ILP's semantic as

`print(if true then 2 else 2.5)` would (after resolution) print `2.0` instead of `2`

Type Unification - 2

What is the type of:

```
if cond then true else ``coucou'' ?
```

Type Unification - 2

What is the type of:

```
if cond then true else ``coucou'' ?
```

Both `bool` and `string` are *absorbing*. Our policy is to be as **permissive** as possible. We arbitrarily decide to choose `bool` in that case.

- `BOOL` when $\tau_1 = \text{BOOL}$ or $\tau_2 = \text{BOOL}$
- `STRING` when $\tau_1 = \text{STRING}$ or $\tau_2 = \text{STRING}$

If-then

There's no void in ILP. We can't force unification. **Policy: permissive**

$$\frac{\Gamma \vdash c : ? \quad \Gamma \vdash e_1 : \tau_1}{\Gamma \vdash \mathbf{if } c \mathbf{ then } e_1 : \tau_1}$$

What happens when `c` evaluates to `false` in unspecified. We'll exploit that later to implement a behaviour that is easy to compile

ILP1's functions

In ILP1, we can only call `print`, `type_of` and `to_string`

These functions can take **anything** but have a defined return type.

- `to_string` return a string
- `type_of` return a string
- `print` returns a boolean

Function Invocation

Let

$$f : (\tau_1, \dots, \tau_n) \rightarrow \tau_r \in \Gamma$$

$$\frac{\Gamma \vdash e_1 : \tau'_1 \quad \dots \quad \Gamma \vdash e_n : \tau'_n}{\Gamma \vdash f(e_1, \dots, e_n) : \tau_r}$$

Remarks:

- f must be a named variable
- Only the return type matters
- The typing algorithm must still visit the arguments to give them type and detect errors!

Formalization of ILP2's type system.

Assignment

- Assignment binds or updates a variable with the type of its expression.
- Follows the interpreter: return type is the type of the initializer.

$$\frac{\Gamma \vdash e : \tau}{\Gamma \vdash x := e : \tau}$$

Effect on environment

$$\Gamma \leftarrow \Gamma[x \mapsto \tau]$$

Loop

$$\frac{\Gamma \vdash c : \tau_c \quad \Gamma \vdash e : \tau_e}{\Gamma \vdash \mathbf{while} \ c \ \mathbf{do} \ e : \mathit{bool}}$$

Remarks:

- Condition is not required to be `bool` here
- Follows the interpreter: return type is `bool`.

Function Definition

What's the type of f ?

```
function f(x)
(
  2 + x
);
```

- What's the return value of $f(1)$?
- What's the return value of $f(1.2)$?
- What's the return value of $f(\text{'coucou'})$?

Function Definition

What's the type of f ?

```
function f(x)
(
  2 + x
);
```

- What's the return value of $f(1)$?
- What's the return value of $f(1.2)$?
- What's the return value of $f(\text{'coucou'})$?

ILP functions can accept several types, they are **polymorph** ...

Function Definition

What's the type of f ?

```
function f(x)
(
  2 + x
);
```

- What's the return value of $f(1)$?
- What's the return value of $f(1.2)$?
- What's the return value of $f(\text{'coucou'})$?

ILP functions can accept several types, they are **polymorph** ...
...or type depends on the argument !

Function Invocation (Primitive)

Idea

If the function is a primitive, defer to the ILP1-typer.

$$\frac{f \notin \text{UserFunctions}}{\Gamma \vdash f(e_1, \dots, e_n) : \tau}$$

Example

- Arithmetic operators
- Built-in functions

Function Invocation (User Defined)

Idea

Functions are specialized based on argument types.

$$\frac{\Gamma \vdash e_1 : \tau_1 \ \dots \ \Gamma \vdash e_n : \tau_n}{\Gamma \vdash f_{\tau_1, \dots, \tau_n}(e_1, \dots, e_n) : \tau_r}$$

Key

- Each call creates or reuses a specialization
- Name mangling encodes argument types: `f_int`, `f_float`, `f_string`

Creating a Function Specialization

Idea

A specialization is created the first time a function is called with a new argument type vector.

- 1 Create a placeholder with unknown return type
- 2 Enter a new scope
- 3 Bind parameters with concrete types
- 4 Type the function body
- 5 Record the inferred return type

This is called **monomorphisation**

Why Monomorphisation?

- No runtime type checks
- Enables aggressive optimization
- Matches C++ templates, Rust generics, ML compilation
- Simplifies backend code generation

Trade-off:

- Code size increase
- Obfuscation of error messages

Typing Recursive Functions

- Functions are typed *on first invocation*
- Typing a function requires typing its body
- Recursive calls require the function type *before* it is known

Typing Recursive Functions

- Functions are typed *on first invocation*
- Typing a function requires typing its body
- Recursive calls require the function type *before* it is known

solution: add a special type PARAM.

- Acts as a *placeholder* for a yet unknown type
- Allows functions to be registered before typing their body (Breaks cyclic dependencies)
- **unifies with every thing**

Intuition

“This function exists, its type will be determined later.”

Example

```
function one(n) {  
    if (n == 0) then 1 else one(n-1)  
}  
one(5)
```

First time one is called with an INT argument:

- Declare return type of one_int as PARAM
- Type its body:
 - ▶ type of the if (n == 0) then 1 else one(n-1)
 - ▶ unify INT and type of one(n-1)
 - ★ type of (n-1) is INT, and we already have a specialization of one for ints
 - ★ return type of one_int is PARAM
 - ▶ unify(INT,PARAM) → INT
 - ▶ update return type of one_int

Restriction

PARAM

lets recursive calls type-check before the real type is known.

- In real-life, several type parameters may be needed, (e.g for mutual recursive function with different return type)
- OCaml has ``a`, ``b`, ``c` ...
- Java has generics `<T1, T2>`

We'll keep it simple and use a single type parameter

Java Implementation

Two new packages:

- `Java/src/com/paracampus/ilp1/treecompiler`
- `Java/src/com/paracampus/ilp2/treecompiler` (inherits from `ilp1/treecompiler`)

In each packages, typed version of the ast node of ILP1 and ILP2.

```
ls ILP1/treecompiler/tast/
```

<code>TASTalternative.java</code>	<code>TASTinteger.java</code>	<code>TASTunaryOperation.java</code>
<code>TASTbinaryOperation.java</code>	<code>TASTinvocation.java</code>	<code>TASTvariable.java</code>
<code>TASTblock.java</code>	<code>TASTprogram.java</code>	<code>Type.java</code>
<code>TASTboolean.java</code>	<code>TASTsequence.java</code>	
<code>TASTfloat.java</code>	<code>TASTstring.java</code>	

A typed AST node: TASTInteger

```
cat ILP1/treecompiler/tast/TASTInteger.java
```

```
public class TASTInteger
    extends ASTInteger implements ITASTInteger {
    private final Type type; // new attribute

    public TASTInteger(String description, Type type) {
        super(description);
        this.type = type;
    }

    @Override
    public Type getType() { return type; }

    @Override
    public <Result, Data, Anomaly extends Throwable>
        Result accept(ITASTvisitor<Result, Data, Anomaly> visitor, Data data)
            throws Anomaly {
        return visitor.visit(this, data);
    }
}
```

Type.java

```
cat ILP1/treecompiler/tast/Type.java
```

```
public enum Type {  
    INT, STRING, FLOAT, BOOL, FUNCTION, PARAM;  
  
    public static Type unify(Type t1, Type t2, String msg)  
        throws TypingException {  
        if (t1 == t2) return t2;  
        if (t1 == PARAM) return t2;  
        if (t2 == PARAM) return t1;  
        if (t1 == BOOL || t2 == BOOL) return BOOL;  
        if (t1 == STRING || t2 == STRING) return STRING;  
        if (t1 == INT && t2 == FLOAT ||  
            t2 == INT && t1 == FLOAT) return FLOAT;  
        throw new TypingException(msg);  
    }  
}
```

Typer as a Visitor

The typer builds a Typed AST (TAST*) from a regular AST.

```
cat ILP1/treecompiler/Typer.java
```

```
public class Typer implements IASTvisitor<ITASTExpression, Void, TypingException> {  
    ...  
  
    public TASTprogram visit(IASTprogram iast) throws TypingException {  
        ITASTExpression body = iast.getBody().accept(this, null);  
        return new TASTprogram(body, body.getType());  
    }  
  
    @Override  
    public ITASTExpression visit(IASTinteger iast, Void context) throws TypingException {  
        return new TASTinteger(iast.getDescription(), Type.INT);  
    }  
  
    ...  
}
```


A Visit Method: Sequence

```
@Override
public ITASTexpression visit(IASTsequence iast, Void context) throws TypingException {
    IASTexpression[] exprs = iast.getExpressions();
    ITASTexpression[] texprs = new ITASTexpression[exprs.length];

    // Type every instruction of the sequence
    for (int i = 0; i < exprs.length; i++) {
        texprs[i] = exprs[i].accept(this, context);
    }

    // return type is the type of the last instruction
    Type lastType = texprs[exprs.length-1].getType();

    // rebuild a sequence with the typed instruction
    return new TASTsequence(texprs, lastType);
}
```

Conclusion

Next week:
Resolution